

Apache Airflow

Orquestación de pipelines de datos como decisión arquitectónica

Ingeniería de Software II — Análisis arquitectónico

Drivers de negocio · Atributos de calidad (ADD) · Trade-offs

| Duración objetivo: 10 min teoría + 5 min demo

Diapositiva 1 — Portada

Apache Airflow: orquestación como decisión de arquitectura

Subtítulo: No "¿qué es?", sino "¿por qué una organización decide adoptarlo y qué cuesta?"

 **Tiempo estimado: 0:20**

Notas del presentador:

Enmarcar la charla: no es una demo de una herramienta de moda, es el análisis de una decisión arquitectónica. La pregunta que guía toda la exposición es *qué driver de negocio justifica incorporar un orquestador y qué atributos de calidad estamos comprando (y pagando)*. Anticipar que cerramos con una demo de 5 minutos.

Diapositiva 2 — El problema de negocio

Cuando la empresa empieza a depender de sus pipelines de datos

Subtítulo: El riesgo no es técnico, es organizacional

- Toda empresa data-driven acumula procesos: ETL nocturnos, cargas a un data warehouse, reentrenamiento de modelos, reportes regulatorios.
- **Fase inicial:** scripts aislados + `cron` + tareas programadas manuales. Funciona... hasta que escala.
- **Síntomas de que el modelo se rompe:**
 - *Dependencias implícitas:* el reporte de las 6 AM asume que la carga de las 5 AM terminó. Nadie lo garantiza.
 - *Fallos silenciosos:* un script falla a las 3 AM y se descubre cuando un gerente abre un dashboard vacío.
 - *Sin reintentos ni recuperación:* un timeout de red obliga a reejecutar todo el

Diapositiva 3 — ¿Qué es Apache Airflow?

Automatización vs. Orquestación

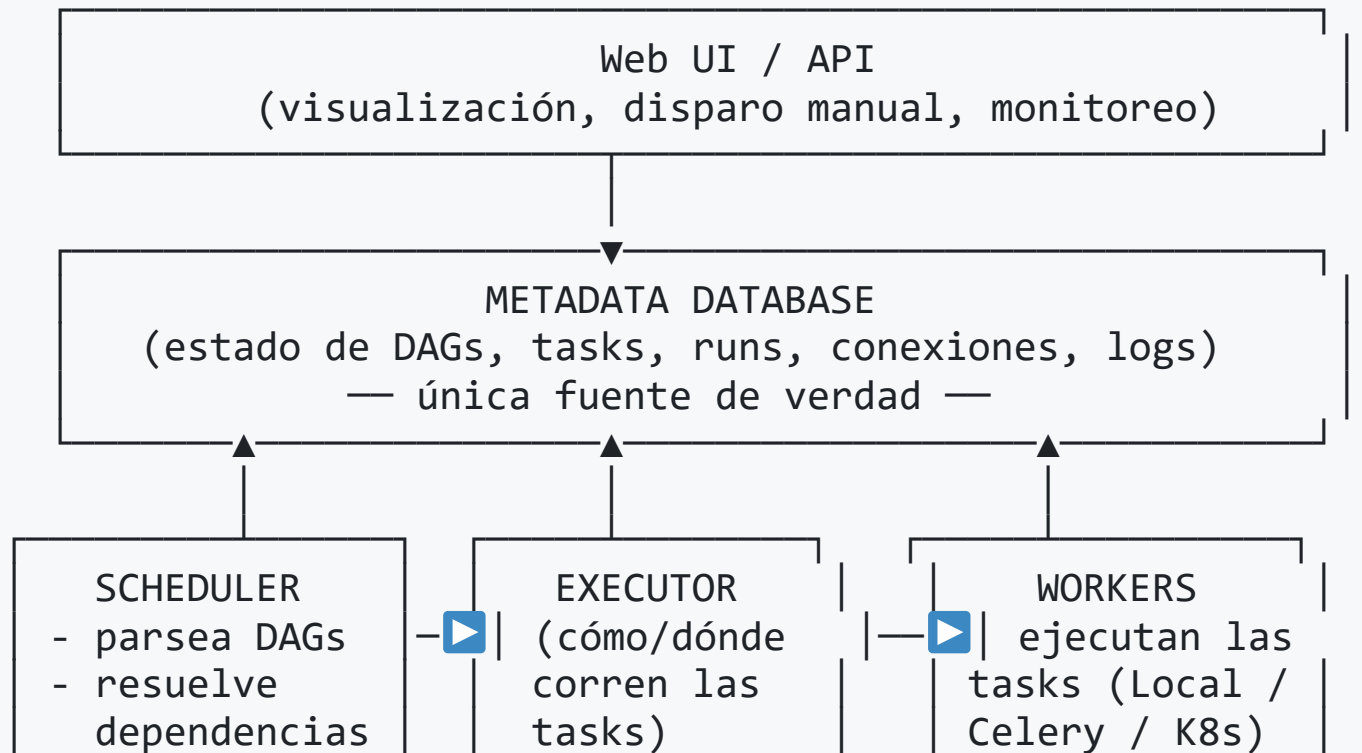
Subtítulo: Plataforma para crear, programar y monitorear flujos de trabajo *como código*

- **Definición:** plataforma open-source para definir flujos de trabajo programáticamente como **DAGs** (grafos dirigidos acíclicos) en Python, y orquestar su ejecución, scheduling y monitoreo.
- **Automatización** = ejecutar una tarea sin intervención humana (lo hace un script en `cron`).
- **Orquestación** = coordinar **muchas tareas interdependientes**: ordenarlas, gestionar dependencias, reintentos, paralelismo, recuperación ante fallos y observabilidad sobre el conjunto.
- **Workflows as Code:** el pipeline es código Python → versionable (Git), revisable (code review), testeable y reproducible

Diapositiva 4 — Arquitectura de Airflow

Componentes y separación de responsabilidades

Subtítulo: Una arquitectura distribuida orientada a desacoplar planificación de ejecución



Diapositiva 5 — Conceptos fundamentales

El vocabulario mínimo de un DAG

Subtítulo: Pocas primitivas, alto poder expresivo

- **DAG:** el flujo completo. *Dirigido y acíclico* → las dependencias tienen sentido y no hay ciclos infinitos.
- **Task:** unidad de trabajo (un nodo del grafo).
- **Operator:** plantilla que define *qué hace* una task. `PythonOperator` , `BashOperator` , `SQLExecuteQueryOperator` , operadores para AWS/GCP, etc.
- **Dependency:** la arista. `validar >> transformar >> cargar` define el orden de ejecución.
- **Scheduling:** cuándo corre el DAG (cron expression o presets, p. ej. `@daily`).
- **Trigger:** qué inicia un run (programado, manual, vía API, o por sensores ante

Diapositiva 6 — La decisión arquitectónica

¿Por qué adoptar una plataforma de orquestación?

Subtítulo: Centralizar una capability transversal en lugar de reimplementarla en cada script

- **El problema es recurrente y transversal:** scheduling, dependencias, reintentos, monitoreo y auditoría reaparecen en *cada* pipeline.
- **Decisión:** ¿lo resolvemos *ad hoc* en cada script o adoptamos una **plataforma** que provea estas capacidades como servicio?
- Adoptar un orquestador = tratar la orquestación como **preocupación arquitectónica de primera clase**, no como detalle de implementación disperso.
- **Alternativas en el espectro:**
 - `cron` + scripts → mínima inversión, nula gobernanza.
 - Orquestadores (Airflow, Prefect, Dagster) → plataforma dedicada

Diapositiva 7 — Atributos de calidad (1/2)

Lo que Airflow fortalece (perspectiva ADD)

Subtítulo: Beneficios concretos sobre los quality attributes

Atributo	Cómo lo aborda Airflow
Fault Tolerance	Reintentos automáticos, estado persistido en Metadata DB, re-ejecución selectiva de tasks fallidas sin rehacer el DAG completo.
Manageability	Web UI con estado de cada run, logs centralizados, instrumentación y métricas (StatsD/OpenTelemetry) para monitoreo y tuning.
Auditability	Historial completo y persistente: qué corrió, cuándo, con qué resultado y por qué falló. Trazabilidad para compliance.
Interoperability	Cientos de <i>providers</i> /operadores: bases de datos, AWS/GCP/Azure,

Diapositiva 8 — Atributos de calidad (2/2)

Los compromisos: ningún atributo es gratis

Subtítulo: Cada beneficio tiene un costo arquitectónico asociado

- **Fault Tolerance tiene un límite:** la Metadata DB es **single point of failure**. Alta disponibilidad real exige HA en la base, scheduler redundante y monitoreo → más complejidad.
- **Scalability cuesta operación:** el `CeleryExecutor` agrega broker (Redis/RabbitMQ); el `KubernetesExecutor` exige operar un clúster. Se gana escala, se paga superficie operativa.
- **Manageability/Auditability requieren disciplina:** la UI ayuda, pero sin convenciones de naming, ownership y alerting el historial se vuelve ruido.
- **Customizability invita al anti-patrón:** como un DAG es Python, es tentador meter lógica pesada de transformación *dentro* de Airflow. Airflow debe **orquestrar no**

Diapositiva 9 — Trade-offs

Qué resuelve y qué NO resuelve Airflow

Subtítulo: Delimitar el alcance evita decisiones arquitectónicas erróneas

✓ Resuelve:

- Orquestación de dependencias complejas entre tareas batch.
- Scheduling confiable, reintentos y recuperación ante fallos.
- Observabilidad, trazabilidad y auditoría de procesos.
- Integración heterogénea (workflows as code).

✗ NO resuelve / NO es:

- No es un motor de procesamiento de datos (eso es Spark/dbt/el warehouse).
- No es streaming en tiempo real — es orientado a *batch* y scheduling. Para eventos continuos → Koflo / Flink.

Diapositiva 10 — Casos de uso reales

Dónde aporta valor

Subtítulo: Patrones de adopción en arquitecturas empresariales

- **ETL / ELT:** orquestar extracción, transformación y carga hacia un Data Warehouse (patrón canónico). En ELT, Airflow dispara las transformaciones (p. ej. dbt) dentro del warehouse.
- **Data Lake / Data Warehouse:** ingesta programada, particionamiento, validaciones de calidad de datos y backfills históricos.
- **Machine Learning Pipelines:** orquestar extracción de features → entrenamiento → validación → despliegue/scoring batch.
- **Automatización de procesos empresariales:** cierres contables, generación de reportes regulatorios, sincronización entre sistemas, conciliaciones nocturnas.

Diapositiva 11 — Comparación con alternativas

Airflow en el ecosistema de orquestación

Subtítulo: Ninguna herramienta es universalmente superior; depende de los drivers

Herramienta	Enfoque	Ventaja	Desventaja vs. Airflow
Cron	Scheduler de tareas	Trivial, ubicuo, cero infra	Sin dependencias, reintentos, UI ni auditoría
Apache NiFi	Dataflow visual (flow-based)	Excelente para streaming/ruteo de datos, low-code	Menos orientado a batch-as-code; otra filosofía
Prefect	Orquestación moderna (Python)	API más liviana, dynamic workflows, DX moderna	Ecosistema/madurez menor; menos providers

Diapositiva 12 — ¿Cuándo usar Airflow?

Recomendado vs. mala elección

Subtítulo: El criterio de decisión en una diapositiva

Buena elección cuando:

- Hay múltiples pipelines batch con dependencias entre tareas.
- Se necesita **scheduling confiable + reintentos + auditoría**.
- Se integran **sistemas heterogéneos** (bases, cloud, APIs).
- El equipo tiene cultura de **infraestructura como código** y Python.

Mala elección cuando:

- El requisito es **procesamiento en tiempo real / streaming**.
- Hay **una sola tarea simple y periódica** → `cron` basta (no pagar el overhead).

Diapositiva 13 — Conclusiones

¿Qué drivers de negocio justifican incorporar Airflow?

Subtítulo: Cierre arquitectónico

- **Confiabilidad de los datos como activo de negocio:** cuando decisiones, reportes y modelos dependen de pipelines, su fiabilidad deja de ser un detalle técnico y pasa a ser un driver de negocio.
- **Gobernanza y auditoría:** trazabilidad de qué corrió, cuándo y con qué resultado → compliance y confianza.
- **Reducción del conocimiento tribal:** workflows as code → versionables, revisables, mantenibles por el equipo.
- **Escalabilidad organizacional:** una plataforma común para decenas/cientos de pipelines en lugar de scripts dispersos.
- **El costo a aceptar:** complejidad operativa y curva de aprendizaje. Airflow se

Diapositiva 14 — Demo

Pipeline diario de reportes de ventas

Subtítulo: De la teoría a un DAG en ejecución (5 min)

Veremos en vivo:

1. Definición de un **DAG** y sus **dependencias**.
2. **Ejecución manual** desde la UI.
3. **Estado de cada task** y logs.
4. Manejo de **errores y retries**.

 **Tiempo estimado: transición — 0:10**

Notas del presentador:

Slide puente hacia la demostración práctica. Mantenerla mínima. (La demo se prepara en una etapa posterior, según el plan.)

Presupuesto de tiempo (teoría)

#	Diapositiva	Tiempo
1	Portada	0:20
2	Problema de negocio	1:15
3	Qué es / Historia	1:00
4	Arquitectura	1:30
5	Conceptos fundamentales	1:00
6	Decisión arquitectónica	0:45
7	Atributos de calidad (1/2)	1:00
8	Atributos de calidad (2/2)	0:50
9	Trade-offs	1:00
10	Conclusiones	0:20